

Actividad Practica Integradora 3

Materia: Logica de programacion

Fecha de entrega: 27/04/25

Alumno: Renzo agustin vallejo - grupo 1

Introducción:

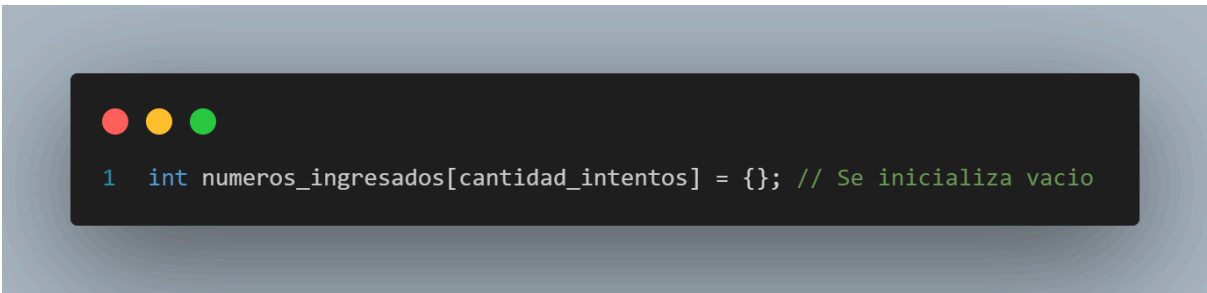
En la siguiente explicación se muestra como se cumple con las consignas de la actividad utilizando lo explicado en las clases anteriores y apoyándome en el material de la lectura 3.

1. Permitir múltiples intentos y guardarlos:

1.1 Crear con código un array para almacenar los números ingresados por el usuario, asegurando que cada intento se guarde correctamente en el array.

Solución:

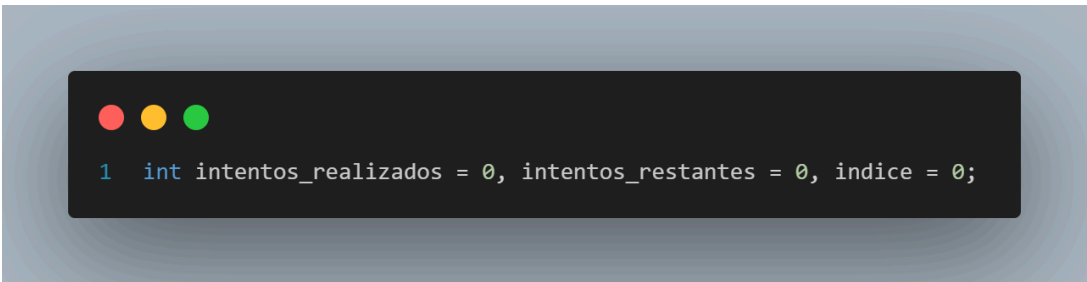
Primero creamos el arreglo donde vamos a guardar todos los números:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a light blue font and shows the declaration and initialization of an array named 'numeros_ingresados' with a size of 'cantidad_intentos', initialized to an empty set of curly braces. A comment in green text explains the initialization.

```
1  int numeros_ingresados[cantidad_intentos] = {}; // Se inicializa vacio
```

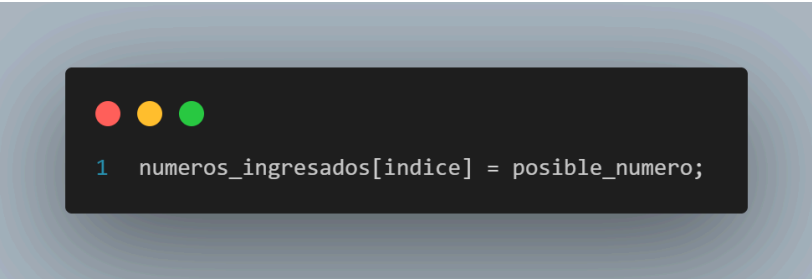
La longitud del arreglo debe ser igual a la cantidad de intentos que haya elegido el usuario, así que utilizamos la variable donde se almacena el valor que el usuario ya introdujo anteriormente.

Luego creamos la variable índice que nos servirá para indicar en qué posición de nuestro arreglo debe guardarse cada número:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a light blue font and shows the declaration and initialization of three integer variables: 'intentos_realizados', 'intentos_restantes', and 'indice', all set to 0.

```
1  int intentos_realizados = 0, intentos_restantes = 0, indice = 0;
```

Luego de que el usuario ingresa por consola un dato, se hacen las validaciones pertinentes: si el dato es de tipo correcto, si está dentro del rango y si no existe en el arreglo, como el arreglo está inicializado vacío, obviamente el primer número ingresado por el usuario será correcto porque no hay número almacenado anteriormente en el arreglo pero luego ya podremos guardarlo en el arreglo como vemos a continuación:



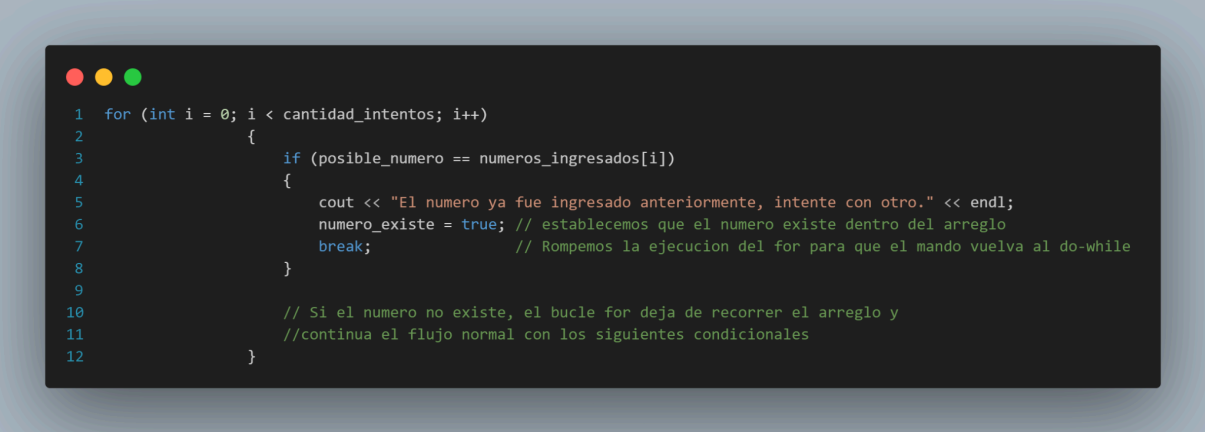
```
1  numeros_ingresados[indice] = posible_numero;
```

2. Verificar duplicados:

2.1 Agregar código que verifique si el número ingresado ya fue intentado anteriormente. Si el número ya fue ingresado, informar al usuario y no contar el intento como válido.

Solución:

Antes de guardar el número ingresado por el usuario en el arreglo, debemos verificar algunas cosas, entre ellas, la más importante es ver si el número ingresado esta dentro del rango, y si esta dentro del rango, se ejecuta un bucle for que corrobora si el número ya existe en el arreglo:



```
1  for (int i = 0; i < cantidad_intentos; i++)
2      {
3          if (posible_numero == numeros_ingresados[i])
4          {
5              cout << "El numero ya fue ingresado anteriormente, intente con otro." << endl;
6              numero_existe = true; // establecemos que el numero existe dentro del arreglo
7              break;               // Rompemos la ejecucion del for para que el mando vuelva al do-while
8          }
9
10         // Si el numero no existe, el bucle for deja de recorrer el arreglo y
11         //continua el flujo normal con los siguientes condicionales
12     }
```

- Para el for se utiliza la variable "cantidad_intentos" con la proposición "i < cantidad_intentos" porque el arreglo se debe recorrer desde el indice 0 hasta (cantidad_intentos - 1).
- Dentro del for tenemos un condicional que por cada iteración del bucle va a corroborar si el "posible_numero" es igual a el número que haya almacenado en el arreglo, para acceder a los indices del arreglo se reutiliza el iterador "i" del for.
- Si el numero ya existe dentro del arreglo se va ejecutar el codigo en el interior de if, donde primero mostramos un mensaje diciendole al usuario que el numero ya se ingreso anteriormente.
- Luego tenemos una variable bool llamada "numero_existe" que inicialmente es declarada false pero si el numero ya existe la cambiamos a true con el fin de poder ejecutar un continue en el bucle do-while para pasar a la siguiente iteración del mismo, ya que si el numero existe el intento no es valido, no podemos almacenarlo en el array y hay que pedirle al usuario que ingrese un numero diferente y como no queremos que el resto del codigo dentro de do-while se siga ejecutando usamos un break dentro del for para romper su ejecucion y devolver el control al do-while para ejecutar el siguiente if con el continue en su interior.

```

1  if (numero_existe) // si numero_existe = true
2      {
3          numero_existe = false; //reiniciamos la variable y la volvemos a false para la proxima iteracion
4          continue; // saltamos a la siguiente iteracion del bucle do-while
5      }

```

se ejecutara solo si el valor de “numero_existe” es true, reiniciamos la variable a false lo cual nos permite seguir usandola para corroborar si los numeros ya fueron ingresados y seguido de eso se ejecuta un continue; para saltar a la siguiente iteración del do-while y que no se ejecute el codigo debajo del if.

3. Mostrar números ingresados:

3.1 Implementar una función que recorra el array de números ingresados y los muestre al usuario, asegurando que los números repetidos no se muestren más de una vez.

Solucion:

Para mostrar los numeros ingresados por el usuario que guarde en el array, en principio lo hice creando un bucle for y recorriendo el arreglo, lo cual funcionaba bien, pero cuando me di cuenta de que tenia que volver a repetir ese mismo bloque en otra parte del codigo pense que lo mejor seria crear una funcion para reutilizar el for sin tener que andar repitiendolo a lo largo del codigo.

```

1  void mostrarNumeros(int* arreglo, int intentos_contados)
2  {
3      cout << "Los numeros que ingresaste fueron: ";
4      // Con el for recorreremos el arreglo para mostrar en pantalla todos los numeros introducidos
5      for (int z = 0; z < intentos_contados; z++)
6      {
7          cout << arreglo[z] << " ";
8      }
9  }

```

Como podemos ver, la función recibe dos parámetros, decidí pasar la referencia del arreglo original como argumento a un puntero para asegurarme de tener los datos correctamente a disposición y poder trabajar con los números guardados allí directamente, lo cual funciona perfecto, luego también paso como argumento los intentos contabilizados como validos porque esa sera la cantidad de iteraciones necesarias para recorrer el arreglo y finalmente mostrar en pantalla los números ingresados por el usuario.